

Principio de Responsabilidad Única

La **S** de SOLID: una clase, una razón para cambiar. El primer paso para escribir software limpio y mantenible.

```
Sordhuoftware :  
{  
  Architectur= BllQ1D"::  
  fsruyande=({ltver"} {  
    algd:tire; (illocucicr){  
    freaineps(Cllertow");  
  }  
  Oblvuader(EM+){  
    sucnovraec(BllL')}{  
    nilfre:(ofd1oK(TLobervit") {  
    fesitt wore=.(dcs/</ftwen Rascif(wmwpdal,lc') ==).{..  
    cuodc(=luve): {  
      stteext:(last-theikie";  
    }  
    vht cahe((Fuprasnetfiod);  
    nal+dnecrâre((lod)) {  
    ndociuc(troptutiasc((Dupalat");  
  }    (.{fforlirchcaure,rcaen" ("'  
    }    (??'({")'  
  }  
  f sfreuseieitrone((an1()) {  
    ErfeatEuhesl:==[coftwaner(8cdoweon") {  
    raves(Vnlnm){  
  }  
  nvaras:(Jles")-G  
  {  
    (Vsatur:(LEV=tiames()Cead')}) --}  
    [rroopurs(/Luafissfl SdY.'O")  
    fioxreus(Hlt( ioleciancer);  
    lss ceba(( ) {  
    Alaruser==ftuhicsn(lp ((CblKroeh),e""}  
  } srs ceidll){  
  } sre iôgô))}  
  } ont src:frur==a((brtExld));  
  }  
  omrseonilucésiny {  
    fealcent"t;  
    rinedbe-efiftotoderodewei1))-2}}  
  }  
  Alisuurde: BllQ1p'):{  
    raments+udt))';  
  }  
}
```

DEFINICIÓN

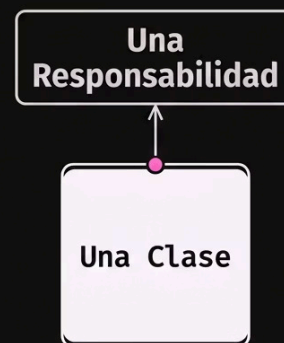
¿Qué es el SRP?

«Una clase debe tener una, y solo una, razón para cambiar.»

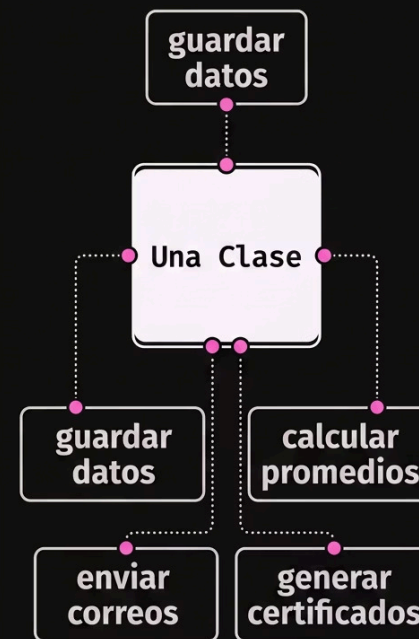
— Robert C. Martin

Esto significa que cada clase debe encargarse de **una sola tarea**. Si una clase hace varias cosas, se vuelve difícil de mantener, probar y reutilizar.

#0F0F10 CORRECTO



#E0D6DE INCORRECTO



ANALOGÍA

El Restaurante como Ejemplo



Cocinero

Solo cocina. No cobra ni reparte.



Camarero

Solo atiende y sirve. No cocina.



Cajero

Solo cobra. No cocina ni atiende.

Si el cocinero también cobrara y repartiera, el restaurante colapsaría. Lo mismo pasa con el software.



Made with GAMMA

¿Cuándo se Incumple el SRP?

Señales de alerta en una clase:

Nombre genérico

Nombres como `Manager` o `Utils` suelen acumular responsabilidades.

Múltiples imports

Si importa clases de correo, base de datos y PDF, hace demasiado.

Métodos heterogéneos

Métodos como `guardar()`, `enviarEmail()` y `generarPDF()` en la misma clase.

Pruebas difíciles

Si necesitas mockear servicios de correo para probar un cálculo, es señal de alerta.

✗ EJEMPLO INCORRECTO

Una Clase que Hace Todo

```
class Estudiante {  
    String nombre;  
    double[] notas;  
  
    void guardarEnBD() { ... }  
    double calcularPromedio() { ... }  
    void enviarEmail() { ... }  
    void generarCertificado() { ... }  
}
```

¿Por qué está mal?

Esta clase tiene **4 responsabilidades**:

- Persistencia (BD)
- Lógica de negocio (promedio)
- Comunicación (email)
- Generación de documentos

Cualquier cambio afecta a las demás.

Responsabilidades Separadas

```
class Estudiante {  
    String nombre;  
    double[] notas;  
}  
  
class CalculadoraPromedios {  
    double calcular(Estudiante e) { ... }  
}  
  
class EmailService {  
    void enviar(Estudiante e) { ... }  
}  
  
class CertificadoService {  
    void generar(Estudiante e) { ... }  
}
```

¿Por qué es mejor?

Cada clase tiene **una sola razón para cambiar**:

- Estudiante: solo datos
- CalculadoraPromedios: solo lógica
- EmailService: solo comunicación
- CertificadoService: solo documentos

BENEFICIOS

¿Por Qué Aplicar el SRP?



Mantenimiento

Los cambios son locales y predecibles.



Legibilidad

El código es más fácil de entender.



Reutilización

Las clases pequeñas se reutilizan mejor.

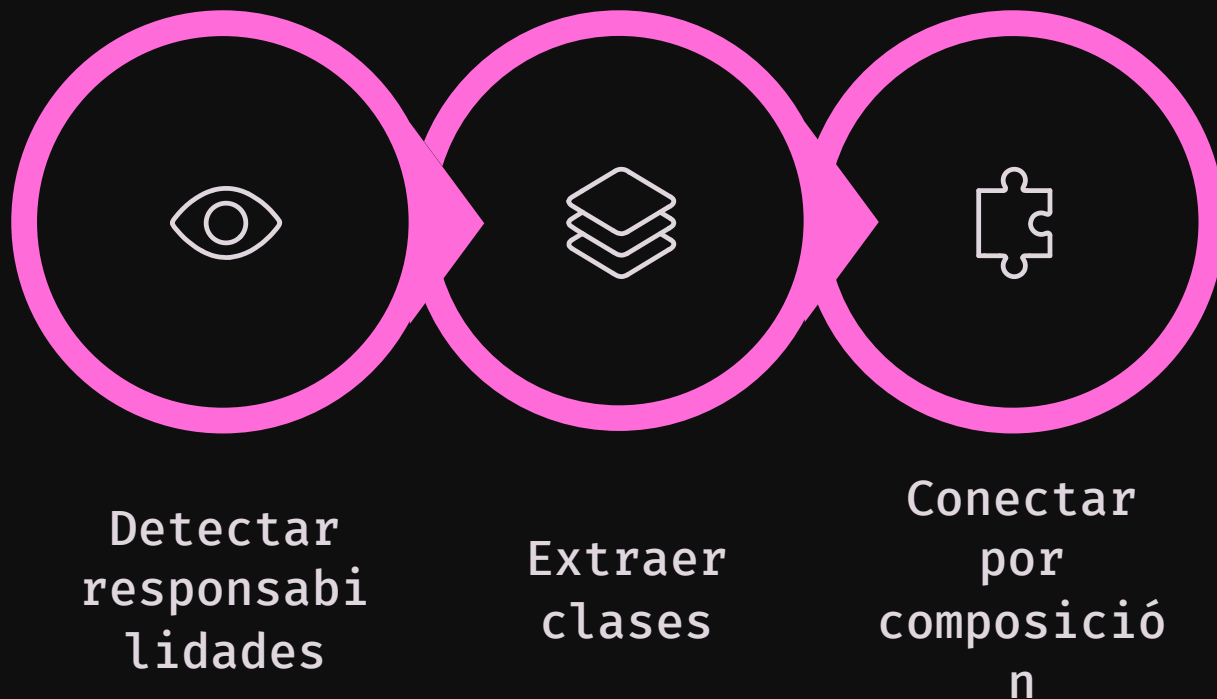


Menos Errores

Menos efectos colaterales al modificar código.



¿Cuándo Aplicar el SRP?



Aplica el SRP desde el inicio del diseño, pero también durante la refactorización. No necesitas perfección inmediata: refactorizar gradualmente es válido.

CONCLUSIÓN

Una Clase, Una Responsabilidad

El SRP es la base de todo buen diseño orientado a objetos. No es una regla rígida, sino una **guía para pensar** en cómo organizar el código.

Recuerda

Si una clase tiene más de una razón para cambiar, divídela.

Próximo paso

Explora la **O de SOLID**: Principio Abierto/Cerrado.

